API Design:

what is API?
- Method (post/get/delete/put) + Path + Body (data) + Headers (metadata)

e.g.: Request Orders:

- method - post
- path /orders
- Headers: content-type: application/json — Authorization: Token
- Body: {...data}
- Response: status code (400, 404, 200, 201, 500) — Headers — Body

Tools used for API Design:
- Swagger: SwaggerEditor — used for documenting the APIs — using **yaml** for writing in the editor

JWT (Json Web Token):

its like an ID, securely represents claims between two parties

structure: Header (Token type + singing algo HMAC, SHA256) . Payload (claims — userInfo + metadata) . Signature (singing the encoded Header and payload with secret key)

AuthenticationService Class :

// method **generateToken**(userID : string) {}

Authentication Middleware:

to access protected routes, a middleware verifies your JWT

the middleware run before protected routes, execute code, modifies req/res/obj, calls next middleware in the stack

checks:

- looks for JWT in authentication headers
- verifies the JWT using **AuthenticationService.verify()**
- extract the userID from the token and attach it to the response obj
- calls next() to proceed to the next middleware
- throws error if JWT is missing

example of using middleware to protects routes:

```
// Example from src/routes/user.route.ts
import express from 'express';
import { authenticate } from '../middleware/auth';
import { UserController } from '../controllers/user.controlle
r';
import asyncHandler from '../middleware/asyncHandler';

const router = express.Router();
const userController = new UserController();

router
  .route('/')
  .get(authenticate, asyncHandler(userController.getAllUsers.
bind(userController)))
  .post(asyncHandler(userController.createUser.bind(userContr
oller)));

router
  .route('/:id')
  .get(authenticate, asyncHandler(userController.getUserById.
bind(userController)))
  .put(authenticate, asyncHandler(userController.updateUser.b
ind(userController)))
  .delete(authenticate, asyncHandler(userController.deleteUse
r.bind(userController)));
```

```
export default router;
```

Role Based Access Control (RBAC):

Setting up the roles for software application, Assign **Users Roles** and give **Permissions**

Role: Category assigned to user — defining the level of access

Permission: Action a user is allowed to perform

Role — Permission Mapping: which permission each role gets

```
// This function creates the 'security guard' middleware
export function hasPermission(permission: Permission) { // Ta
kes the required permission (e.g., READ_ORDER)
    return (req: Request, res:Response, next: NextFunction) =
> { // Returns the actual middleware function
        // 1. Get the user's role from the request (assume th
ey are logged in)
        const userRole = req.user.role;

        // 2. Check if the user's role has the required permi
ssion in our mapping
        if (!rolePermission[userRole].includes(permission)) {
            // 5. If not, throw an error (Access Denied!)
            throw new InsufficientPermissionException();
        }

        // 4. If yes, call next() to allow the request to pro
ceed
        next();
    }
}
```

API Architecture:

Authentication: require Authorization Bearer token

Identity: are never accepted from the client request bodies. the backend derives userID and role from JWT

format: application/json

Client Layer ———— Business Layer (API logic) ———— Database Layer (persistence)

In Creating User for example

- sends User Data
- Validate Request Data/ Assign Default Role
- Create User Record in Database
- User Created Successfully + Generate JWT (userID + Role)
- Send status code 201 to client layer + JWT
- client saves the JWT for future requests

In Creating a course:

- client side send post/course + JWT
- API Interpret JWT + verify identity
- RBAC check
- if user not admin — 403 status code to client layer
- else — save course with adminID
- record saved in Database layer
- status code 201 created

API definitions for Invoice Management System

1. Brainstorming:

Authentication:

`POST /api/v1/auth/login` — FR001-002-003

`POST /api/v1/auth/logout` — FR-004

Dashboard:

`GET /api/v1/dashboard` — FR006-FR011 (all display) one endpoint cleaner

Customer Management

creation, editing, archiving, archive guards, name searching, status filtering, and viewing customer invoices.

`POST /api/v1/customers` — FR012

`PATCH /api/v1/customers/{customerId}` — FR013

`POST /api/v1/customers/{customerId}/archive` — FR014-015

`GET /api/v1/customers` `GET /api/v1/customers/{customerId}` — FR016-017

`GET /api/v1/customers/{customerId}/invoices` — FR018

For Search : `GET /customers?` `search=acme&status=active`

Contract Management

`POST /api/v1/contracts` — FR-019, FR-022

`PATCH /api/v1/contracts/{contractId}` — FR020

`POST /api/v1/contracts/{contractId}/archive` — FR021

`GET /api/v1/contracts` `GET /api/v1/contracts/{contractId}` — List Contracts / contract details

`GET /api/v1/customers/{customerId}/contracts` — Contracts belonging to customer

FR-025 scheduled contract-expiry check it should be internal service and not API

Invoice Management

`POST /api/v1/invoices` — FR-026, FR-031–034

`PATCH /api/v1/invoices/{invoiceId}` — FR-028, FR-031–033

`POST /api/v1/invoices/{invoiceId}/issue` — FR-027

`POST /api/v1/invoices/{invoiceId}/cancel` — FR-030

`GET /api/v1/invoices/{invoiceId}/pdf` — FR035

`POST /api/v1/invoices/{invoiceId}/send` — FR036 , FR-037

`GET /api/v1/invoices` `GET /api/v1/invoices/{invoiceId}` — List details, search and filters (FR-062 - 065 search module)

for FR-029 we will treat it as a guard and same for calculations — no end points API

Invoice Delivery

`GET /api/v1/invoices/{invoiceId}/deliveries` — View invoice delivery history, associated with FR-037 (Potential API)

Payment Management

`POST /api/v1/invoices/{invoiceId}/payments` — FR-046–049

`GET /api/v1/invoices/{invoiceId}/payments` — FR-050 for payment history

`GET /api/v1/payments?from=...&to=…` — reporting/payment views.

Reminder System

`GET /api/v1/settings/reminders` — Retrieve configured reminder schedule

`PUT /api/v1/settings/reminders` — Change reminder offsets

`GET /api/v1/invoices/{invoiceId}/reminders` — FR-054

FR-052, FR-053, FR-055 Generate/send due reminders automatically will not have API endpoints, they are background services

Administrator Notifications

`GET /api/v1/notifications`

`GET /api/v1/notifications/{notificationId}`

Reports

`GET /reports/financial-summary` — FR-056, FR-058

`GET /reports/customer-balances` — FR-057

`GET /reports/payments` — FR-059

all the above API must have a query parameters later

Settings

`GET /api/v1/settings/business`
`PUT /api/v1/settings/business`

`GET /api/v1/settings/invoice`
`PUT /api/v1/settings/invoice`

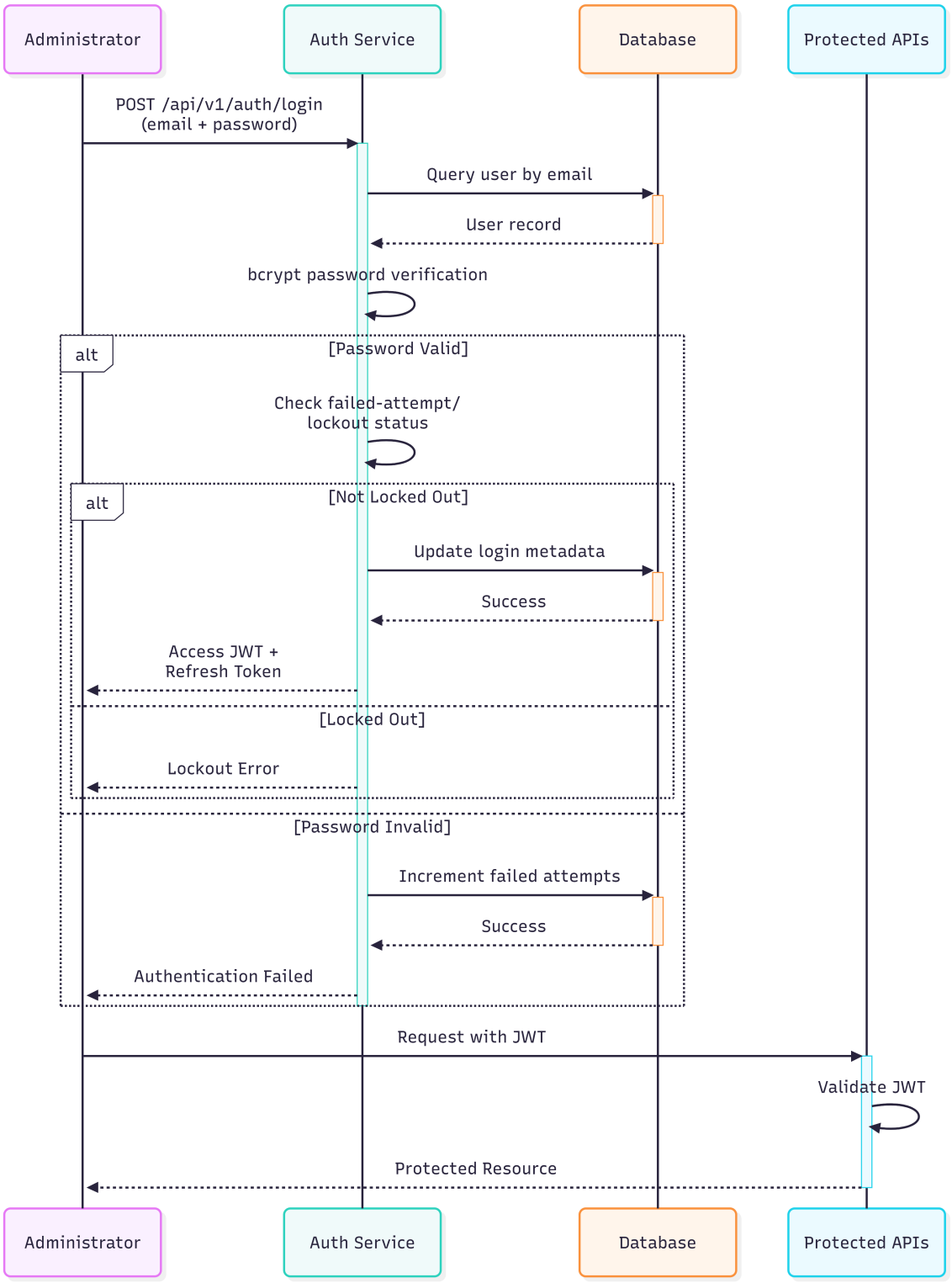
`GET /api/v1/settings/reminders`
`PUT /api/v1/settings/reminders`

`GET /api/v1/settings/notifications`
`PUT /api/v1/settings/notifications`

`GET /api/v1/settings/email-templates`
`PUT /api/v1/settings/email-templates/{templateType}`

2. Sketching

Authentication



`POST /api/v1/auth/refresh` — Obtain a new access JWT

Access Token

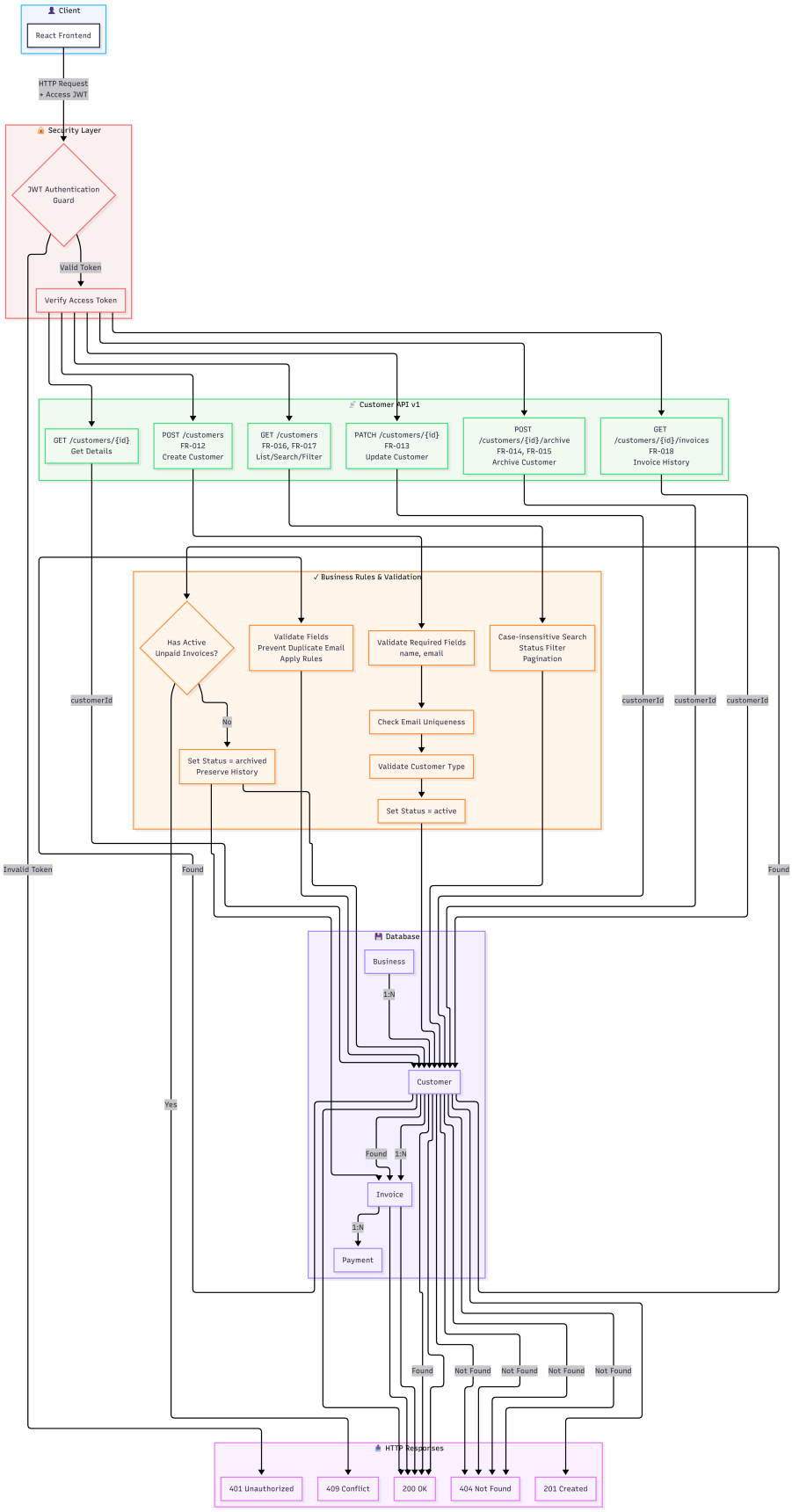
- JWT
- short lifetime
- used for API authorization

Refresh Token

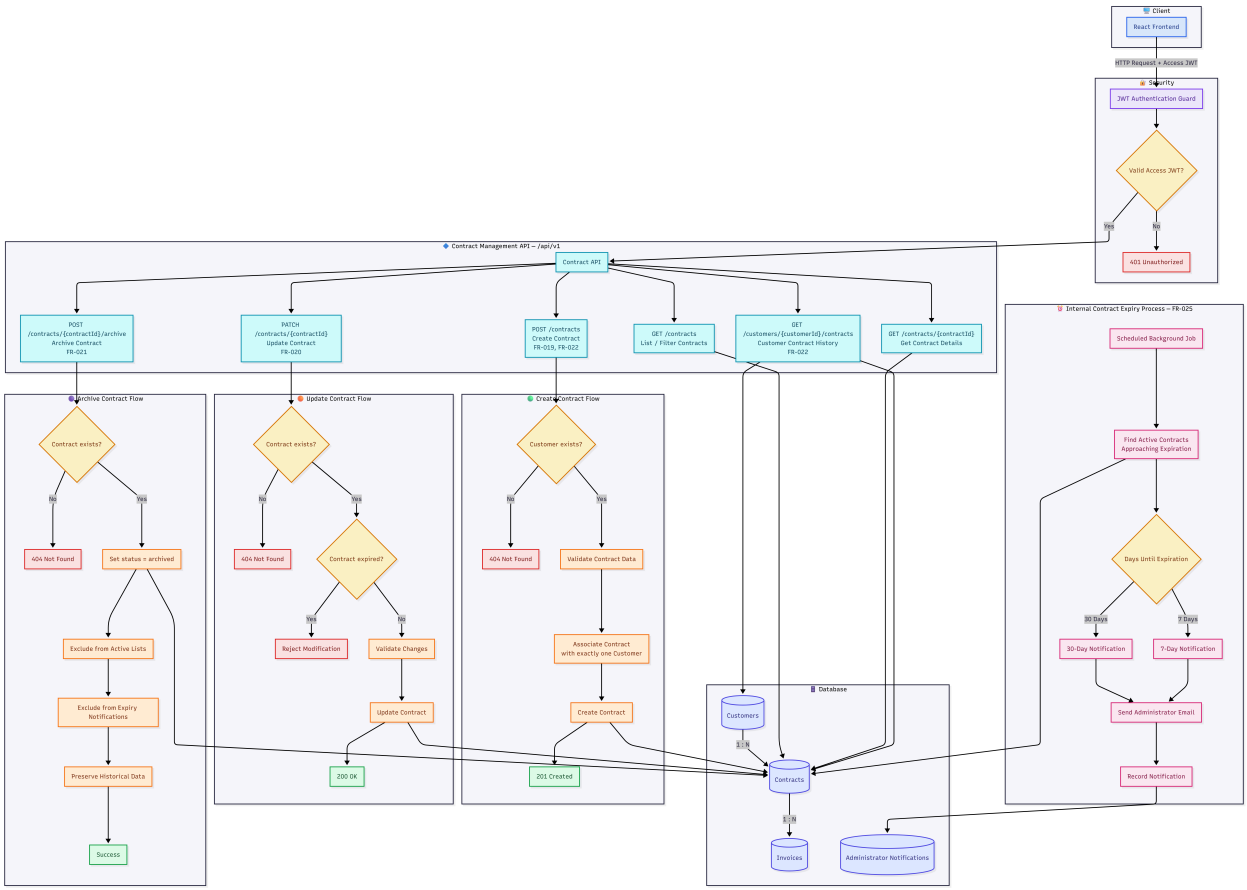
- longer lived
- revocable
- preferably stored in Secure + HttpOnly cookie
- associated with a server-side session

Access tokens should only be valid for a short period.

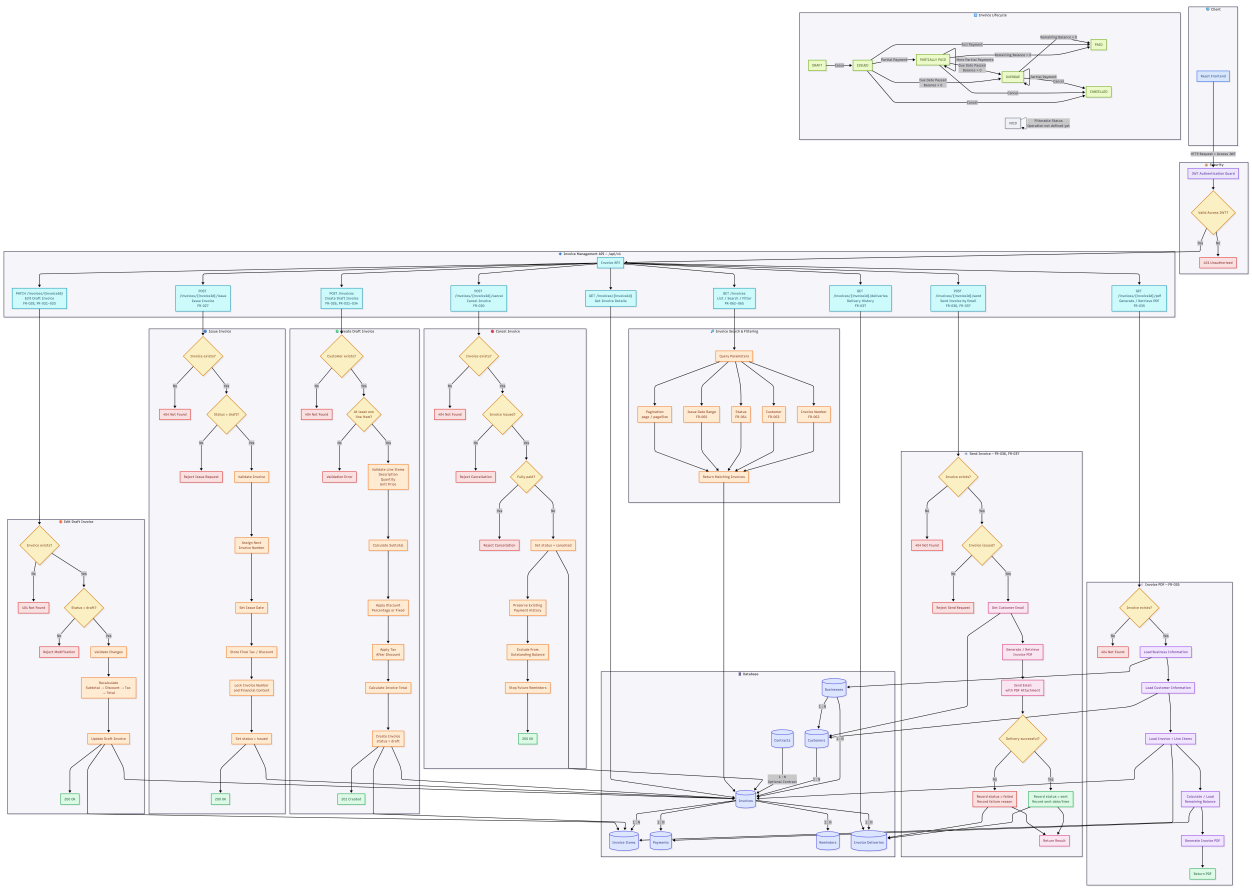
Customer


Client
React Frontend
HTTP Request + Access JWT
Security Layer
JWT Authentication Guard
Valid Token
Verify Access Token
Customer API v1
GET /customers/{id} Get Details
POST /customers FR-012 Create Customer
GET /customers FR-016, FR-017 List/Search/Filter
PATCH /customers/{id} FR-013 Update Customer
POST /customers/{id}/archive FR-014, FR-015 Archive Customer
GET /customers/{id}/invoices FR-018 Invoice History
✓ Business Rules & Validation
Has Active Unpaid Invoices?
Validate Fields Prevent Duplicate Email Apply Rules
Validate Required Fields name, email
Case-insensitive Search Status Filter Pagination
No
Set Status = archived Preserve History
Check Email Uniqueness
Validate Customer Type
Set Status = active
customerId
customerId
customerId
customerId
Invalid Token
Found
Found
Database
Business
1:N
Customer
Yes
Found
1:N
Invoice
1:N
Payment
Found
Not Found
Not Found
Not Found
Not Found
HTTP Responses
401 Unauthorized
409 Conflict
200 OK
404 Not Found
201 Created

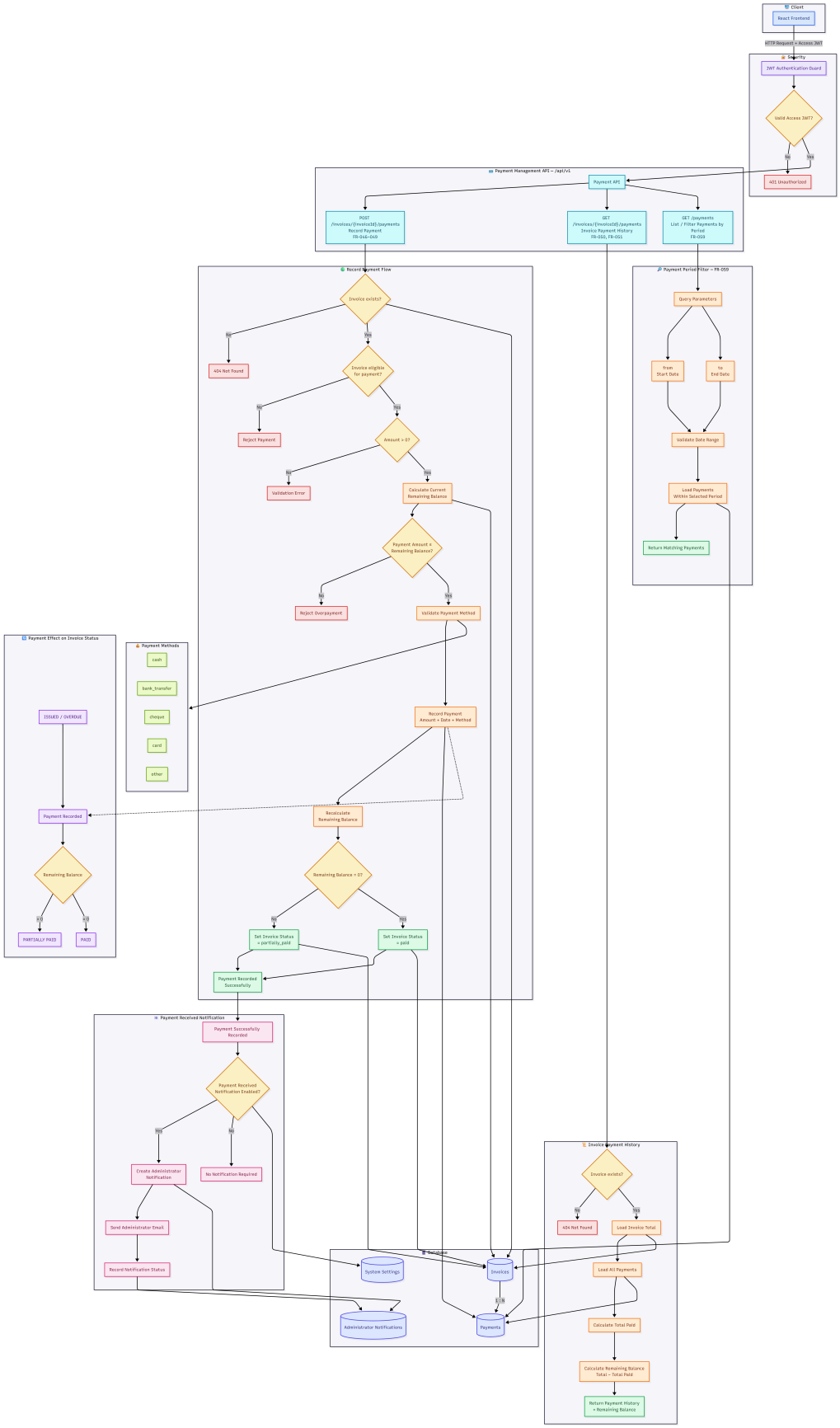
Contract



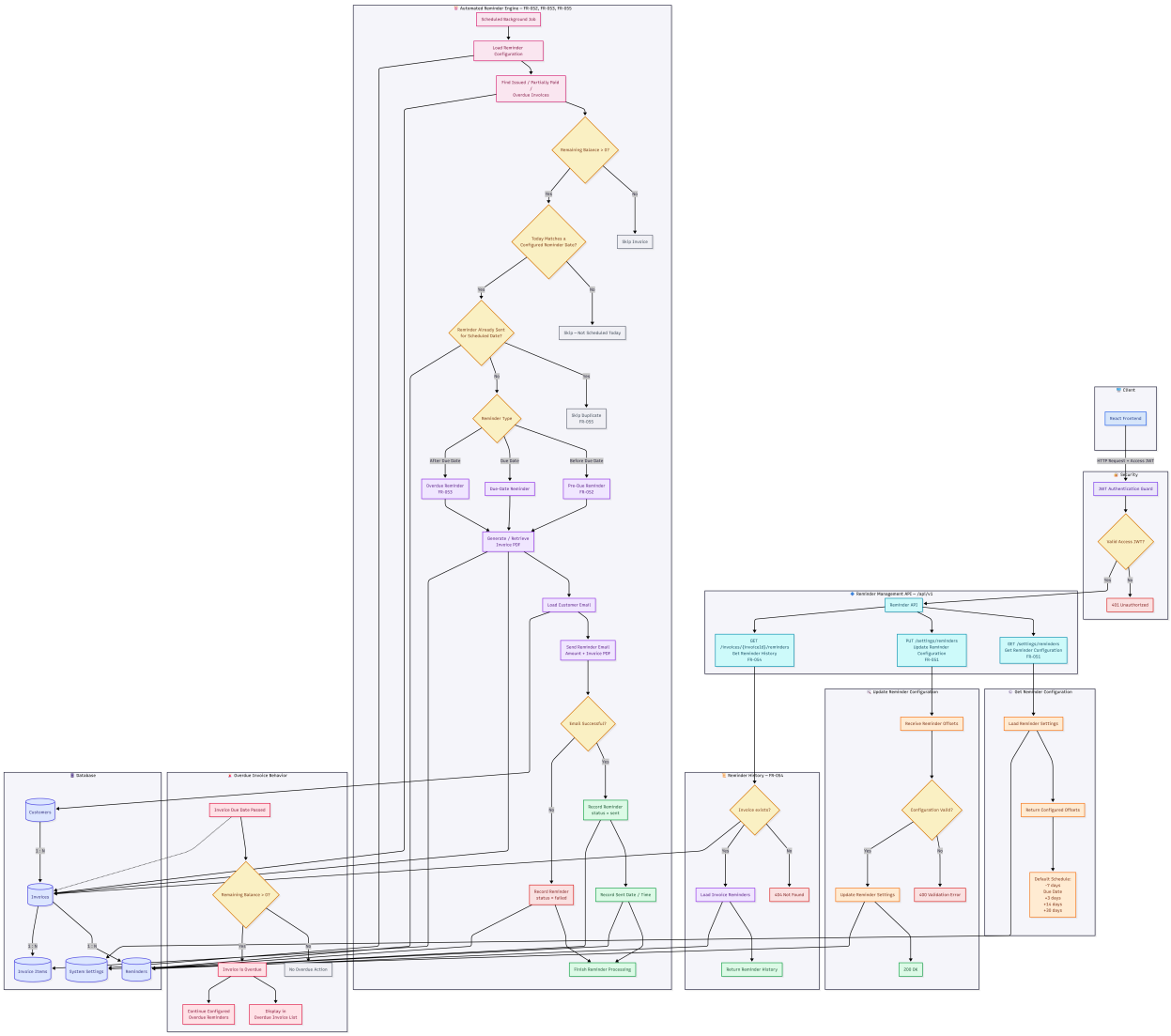
Invoice



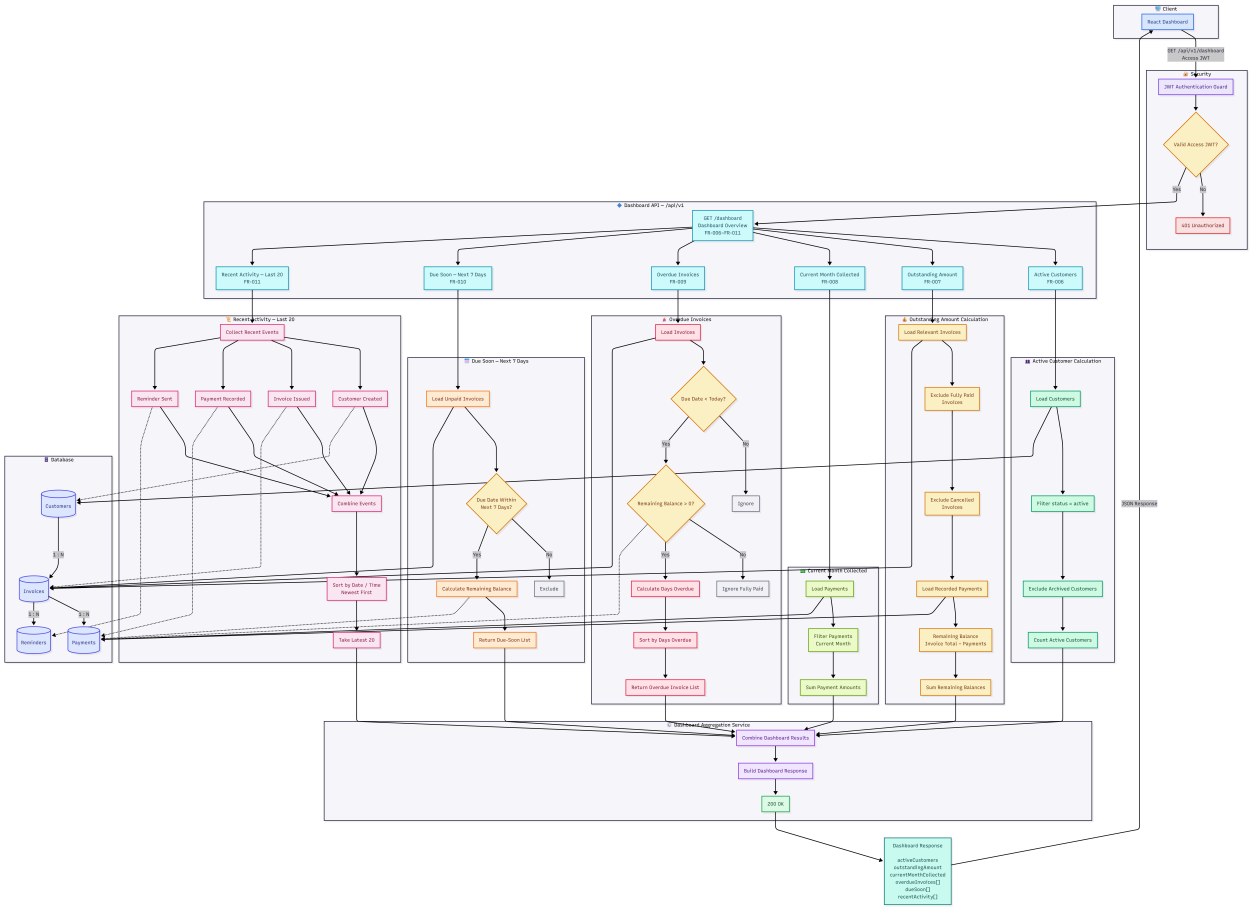
Payment


Client
React Frontend
HTTP Request + Access JWT
Security
JWT Authentication Guard
Valid Access JWT?
No
Yes
401 Unauthorized
Payment Management API — /api/v1
Payment API
POST /invoices/{invoiceId}/payments Record Payment FR-046–049
GET /invoices/{invoiceId}/payments Invoice Payment History FR-050, FR-051
GET /payments List / Filter Payments by Period FR-058
Record Payment Flow
Invoice exists?
No
404 Not Found
Yes
Invoice eligible for payment?
No
Reject Payment
Yes
Amount > 0?
No
Validation Error
Yes
Calculate Current Remaining Balance
Payment Amount ≤ Remaining Balance?
No
Reject Overpayment
Yes
Validate Payment Method
Payment Period Filter — FR-059
Query Parameters
from Start Date
to End Date
Validate Date Range
Load Payments Within Selected Period
Return Matching Payments
Payment Effect on Invoice Status
ISSUED / OVERDUE
Payment Recorded
Remaining Balance
≤ 0
PARTIALLY PAID
≤ 0
PAID
Payment Methods
cash
bank_transfer
cheque
card
other
Record Payment Amount + Date + Method
Recalculate Remaining Balance
Remaining Balance = 0?
No
Set Invoice Status = partially_paid
Yes
Set Invoice Status = paid
Payment Recorded Successfully
Payment Received Notification
Payment Successfully Recorded
Payment Received Notification Enabled?
Yes
Create Administrator Notification
Send Administrator Email
Record Notification Status
No
No Notification Required
Database
System Settings
Invoices
1 : N
Administrator Notifications
Payments
Invoice Payment History
Invoice exists?
No
404 Not Found
Yes
Load Invoice Total
Load All Payments
Calculate Total Paid
Calculate Remaining Balance Total − Total Paid
Return Payment History + Remaining Balance

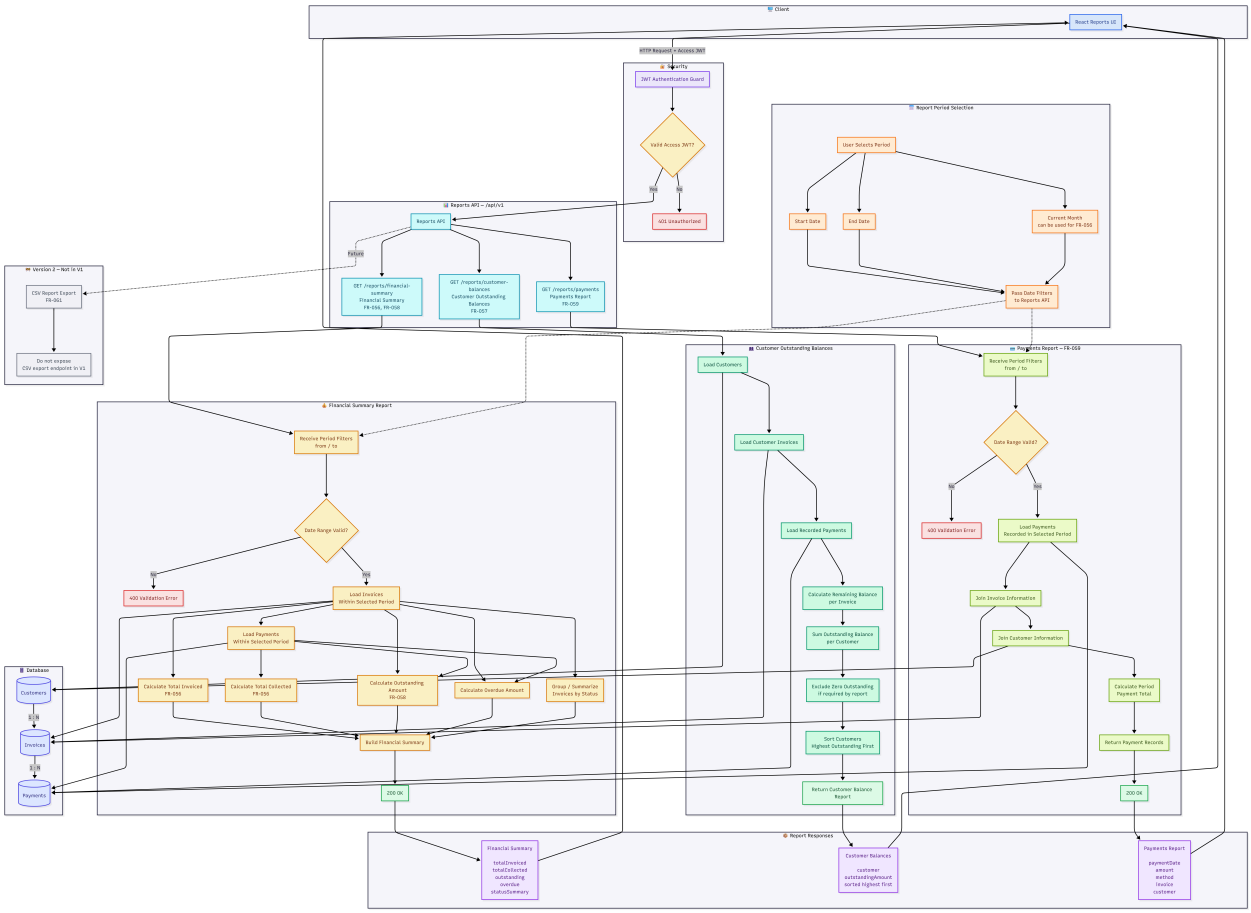
Reminder



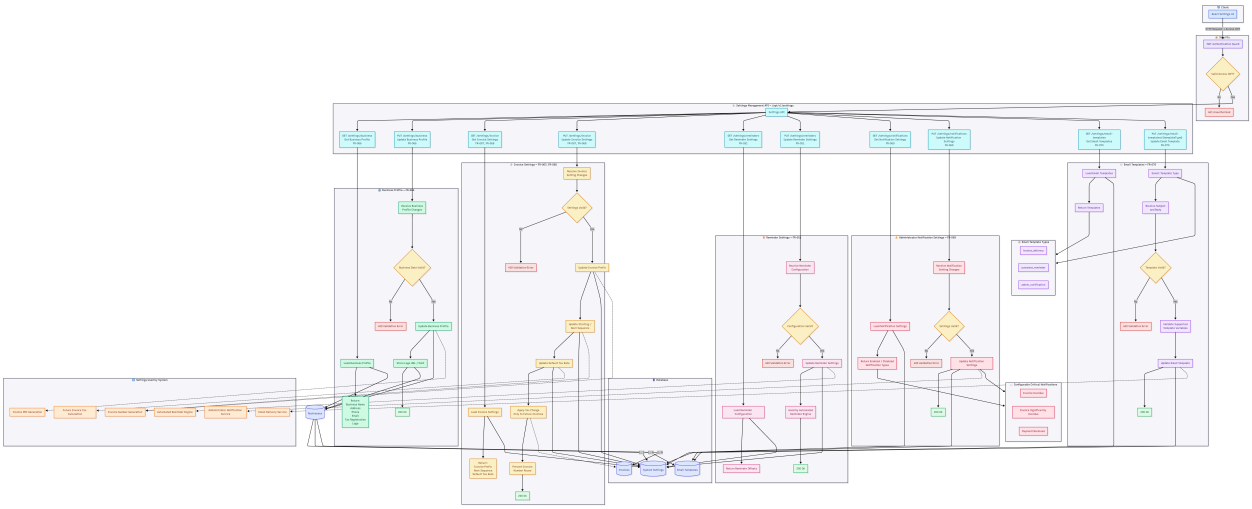
Dashboard


Client
React Dashboard
GET /api/v1/dashboard
Access JWT
Security
JWT Authentication Guard
Valid Access JWT?
Yes
No
401 Unauthorized
Dashboard API – /api/v1
GET /dashboard
Dashboard Overview
FR-006–FR-011
Recent Activity – Last 20
FR-011
Due Soon – Next 7 Days
FR-010
Overdue Invoices
FR-009
Current Month Collected
FR-008
Outstanding Amount
FR-007
Active Customers
FR-006
Recent Activity – Last 20
Collect Recent Events
Reminder Sent
Payment Recorded
Invoice Issued
Customer Created
Combine Events
Sort by Date / Time
Newest First
Take Latest 20
Due Soon – Next 7 Days
Load Unpaid Invoices
Due Date Within
Next 7 Days?
Yes
No
Calculate Remaining Balance
Exclude
Return Due-Soon List
Overdue Invoices
Load Invoices
Due Date < Today?
Yes
No
Ignore
Remaining Balance > 0?
Yes
No
Ignore Fully Paid
Calculate Days Overdue
Sort by Days Overdue
Return Overdue Invoice List
Current Month Collected
Load Payments
Filter Payments
Current Month
Sum Payment Amounts
Outstanding Amount Calculation
Load Relevant Invoices
Exclude Fully Paid
Invoices
Exclude Cancelled
Invoices
Load Recorded Payments
Remaining Balance
Invoice Total − Payments
Sum Remaining Balances
Active Customer Calculation
Load Customers
Filter status = active
Exclude Archived Customers
Count Active Customers
Database
Customers
1 : N
Invoices
1 : N
1 : N
Reminders
Payments
Dashboard Aggregation Service
Combine Dashboard Results
Build Dashboard Response
200 OK
JSON Response
Dashboard Response
activeCustomers
outstandingAmount
currentMonthCollected
overdueInvoices[]
dueSoon[]
recentActivity[]

Reports


Client
React Reports UI
HTTP Request + Access JWT
Security
JWT Authentication Guard
Valid Access JWT?
Yes
No
401 Unauthorized
Report Period Selection
User Selects Period
Start Date
End Date
Current Month can be used for FR-056
Pass Date Filters to Reports API
Reports API – /api/v1
Reports APIs
Future
GET /reports/financial-summary Financial Summary FR-056, FR-058
GET /reports/customer-balances Customer Outstanding Balances FR-057
GET /reports/payments Payments Report FR-059
Version 2 – Not in V1
CSV Report Export FR-061
Do not expose CSV export endpoint in v1
Financial Summary Report
Receive Period Filters from / to
Date Range Valid?
No
Yes
400 Validation Error
Load Invoices Within Selected Period
Load Payments Within Selected Period
Calculate Total Invoiced FR-056
Calculate Total Collected FR-056
Calculate Outstanding Amount FR-058
Calculate Overdue Amount
Group / Summarize Invoices by Status
Build Financial Summary
200 OK
Customer Outstanding Balances
Load Customers
Load Customer Invoices
Load Recorded Payments
Calculate Remaining Balance per Invoice
Sum Outstanding Balance per Customer
Exclude Zero Outstanding If required by report
Sort Customers Highest Outstanding First
Return Customer Balance Report
Payments Report – FR-059
Receive Period Filters from / to
Date Range Valid?
400 Validation Error
Load Payments Recorded in Selected Period
Join Invoice Information
Join Customer Information
Calculate Period Payment Total
Return Payment Records
200 OK
Database
Customers
1 : N
Invoices
1 : N
Payments
Report Responses
Financial Summary
totalInvoiced
totalCollected
outstanding
overdue
statusSummary
Customer Balances
customer
outstandingAmount
sorted highest first
Payments Report
paymentDate
amount
method
invoice
customer

Settings



Administration Notification

